Sorting Algorithm -

- Sorting = arranging data in a specific order (ascending/descending).
- Helps in faster searching, data organization, and efficient use in other algorithms (like Binary Search, Greedy, Divide & Conquer).

Real-Life Examples:

- Sorting names alphabetically.
- Sorting prices on e-commerce websites.
- Arranging exam scores in ascending order.

Sorting Algorithm -

Types Of Sorting -

- Simple / Basic Sorting
  - Bubble Sort
  - Selection Sort
  - Insertion Sort
- Efficient Sorting
  - Merge Sort (O(n log n))
  - Quick Sort (O(n log n))
  - Heap Sort
- Special Cases
  - Counting Sort
  - Radix Sort.

Bubble Sort -

Algorithm (Step-by-step):

1. Compare adjacent elements in the array.
2. If they are in the wrong order → swap them.
3. Repeat the process for all elements until the array is sorted.
4. After each pass, the largest element "bubbles up" to its correct position.

Time Complexity:

- Best case: O(n) (already sorted, optimized version with no swaps)
- Worst case: $O(n^2)$
- Average case: $O(n^2)$
- Space Complexity: O(1) (in-place sorting)

Bubble Sort -

Ques. Sort an array using Bubble Sort.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  # swap
                swapped = True
        if not swapped:  # optimization: stop if no swaps
            break
    return arr

arr = [64, 34, 25, 12, 22, 11, 90]
print("Sorted Array:", bubble_sort(arr))
```

Selection Sort -

Algorithm (Step-by-step):

1. Divide the array into two parts → *sorted* and *unsorted*.
2. Repeatedly find the minimum element from the unsorted part.
3. Swap it with the first element of the unsorted part.
4. Expand the sorted part by one, until the whole array is sorted.

Time Complexity:

- Best case: $O(n^2)$
- Worst case: $O(n^2)$
- Average case: $O(n^2)$
- Space Complexity: O(1) (in-place sorting)

Selection Sort -

Ques. Sort an array using Selection Sort.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  # swap
    return arr

arr = [64, 25, 12, 22, 11]
print("Sorted Array:", selection_sort(arr))
```